*PES University — Dept. of CSE*

# Lab 1: UML Use-Case Model

## *Problem Statement #23 — Hyperlocal Courier Dispatch & Tracking Engine*

This document is the companion Use-Case Model to the UML Use-Case Diagram. It sets out, in full, the actors that interact with the Hyperlocal Courier Dispatch & Tracking Engine, a brief description of every use case shown on the diagram, and the reasoning behind each «include»/«extend» relationship, before presenting the diagram itself.

### 1. Actor Profiles

| Actor | Goals | Responsibilities / Interactions |
|---|---|---|
| Sender Client | Get a parcel picked up and delivered quickly, with visibility into its progress. | Creates delivery requests (pickup/drop-off, package details); views live rider location and status; receives delivery confirmation; may report a delay or rate the delivery afterwards. |
| Delivery Rider | Fulfil as many deliveries as possible, efficiently and safely, within service commitments. | Receives dispatch notifications; accepts or rejects jobs within the response window; follows the system's optimized multi-stop route; confirms delivery by entering the recipient's OTP. |
| Recipient | Receive the correct parcel and confirm receipt with minimal friction. | Receives the OTP generated at dispatch (via SMS/app); shares it with the rider at the point of delivery to authorize the 'Delivered' status change. |

## 2. Use-Case Brief Descriptions

Every use case shown in the diagram is described below in brief-description format: primary actor, secondary actor (if any), the trigger that starts it, its precondition, a short narrative, and the requirement it realizes.

| **UC-01: Create Delivery Request** | | | |
|---|---|---|---|
| Primary Actor | Sender Client | Secondary Actor | — |
| Trigger | Sender opens the app/site and chooses "New Delivery". | Precondition | Sender is logged in with a valid account. |
| Description | The sender specifies pickup location, drop-off location, recipient contact details, and package details. On submission the system saves the request with a unique tracking ID and status 'Pending Match', then always triggers UC-02 (Match Nearest Rider) as an included step. | | |
| Related Requirement | FR-002 | | |

| **UC-02: Match Nearest Rider** | | | |
|---|---|---|---|
| Primary Actor | Sender Client | Secondary Actor | System (automated) |
| Trigger | Included automatically whenever UC-01 completes. | Precondition | At least one delivery rider is marked active within the service area. |
| Description | The system searches for active riders within a 3 km radius of the pickup point and assigns the request to the nearest eligible rider, then hands off to UC-03 for the rider's response. | | |
| Related Requirement | FR-001 | | |

| **UC-03: Accept / Reject Dispatch** | | | |
|---|---|---|---|
| Primary Actor | Delivery Rider | Secondary Actor | — |
| Trigger | Rider receives a dispatch notification from UC-02. | Precondition | Rider's app is online and able to receive push notifications. |
| Description | The rider is shown the pickup location and package summary and has 30 seconds to accept or reject. On rejection or timeout, the system automatically reassigns the request to the next-nearest available rider (re-invoking UC-02). | | |
| Related Requirement | FR-003 | | |

| **UC-04: Optimize Multi-Stop Route** | | | |
|---|---|---|---|
| Primary Actor | Delivery Rider | Secondary Actor | — |
| Trigger | Rider has more than one active accepted delivery at the same time. | Precondition | Rider holds 2 or more accepted, undelivered parcels. |
| Description | The system computes a stop sequence that minimizes total travel distance across all of the rider's pending deliveries and displays it as a suggested route the rider can follow. | | |
| Related Requirement | FR-005 | | |

| **UC-05: Complete Delivery** | | | |
|---|---|---|---|
| Primary Actor | Delivery Rider | Secondary Actor | Recipient |
| Trigger | Rider arrives at the drop-off location and taps "Complete Delivery". | Precondition | Order status is 'Out for Delivery'. |

| | |
|---|---|
| Description | The rider initiates delivery completion, which always includes UC-06 (Verify OTP) as a mandatory sub-step — a delivery can never be marked complete without a successful OTP check. |
| Related Requirement | FR-004 |

| **UC-06: Verify OTP** | | | |
|---|---|---|---|
| Primary Actor | Delivery Rider | Secondary Actor | Recipient |
| Trigger | Included automatically whenever UC-05 executes. | Precondition | A dispatch-time OTP exists for the order and has been delivered to the recipient. |
| Description | The recipient shares the OTP shown on their device with the rider, the rider enters it, and the system validates it before allowing the order status to change to 'Delivered'. | | |
| Related Requirement | FR-004 | | |

| **UC-07: Track Delivery Status** | | | |
|---|---|---|---|
| Primary Actor | Sender Client | Secondary Actor | — |
| Trigger | Sender opens the tracking screen for an active order. | Precondition | Order status is anything other than 'Delivered' or 'Cancelled'. |
| Description | The sender views the rider's live GPS position and current order status, refreshed with under 2-second latency. May optionally lead into UC-08 if the sender perceives a delay. | | |
| Related Requirement | NFR-001 | | |

| **UC-08: Report Delivery Delay** | | | |
|---|---|---|---|
| Primary Actor | Sender Client | Secondary Actor | — |
| Trigger | Sender, while tracking (UC-07), judges the delivery to be later than expected. | Precondition | Order is still in progress and being tracked. |
| Description | An optional extension of UC-07: the sender flags the order for support attention. This only fires conditionally, which is why it is modelled as an «extend» rather than an «include». | | |
| Related Requirement | extends UC-07 | | |

| **UC-09: Rate & Provide Feedback** | | | |
|---|---|---|---|
| Primary Actor | Sender Client | Secondary Actor | — |
| Trigger | Order status changes to 'Delivered'. | Precondition | UC-05 (Complete Delivery) has finished successfully. |
| Description | The sender rates the rider and delivery experience and may leave a comment, closing the request lifecycle. | | |
| Related Requirement | Supporting UC | | |

## 3. Include / Extend Relationships

Per UML notation, «include» denotes mandatory sub-behaviour that always executes as part of the base use case (used for reuse), while «extend» denotes optional behaviour that only executes under a condition (used for variation). The table below justifies each relationship used in the diagram on the following page.

| Relationship | Justification |
|---|---|
| UC-01 Create Delivery Request «include» UC-02 Match Nearest Rider | Rider matching always runs as part of creating a request — mandatory sub-behaviour, hence include. |
| UC-05 Complete Delivery «include» UC-06 Verify OTP | A delivery can never be marked complete without OTP validation — mandatory sub-behaviour, hence include. |
| UC-08 Report Delivery Delay «extend» UC-07 Track Delivery Status | Reporting a delay is optional and only triggered conditionally while tracking — hence extend. |

### 4. UML Use-Case Diagram — see following page.

# Hyperlocal Courier Dispatch & Tracking Engine — Use-Case Diagram